



نام دانشجو :

رضا ندیمی

شماره دانشجویی:

832204022

دانشکده سامانه های هوشمند - مکترونیک

تمرینات مکترونیک 2 سری هفتم

تاریخ:

1405/03/28

استاد مربوطه:

دکتر تارویردی زاده

فهرست

- فضای توسعه‌ی مورد استفاده با توجه به عدم دسترسی به مخازن نرم افزار بین المللی 3
- 1- با فعال کردن ماژولهای تایمر و وقفه در میکروکنترلر، یک سیگنال PWM با فرکانس 50 Hz و Duty Cycle برابر با ۵۰٪ در یکی از پایه های میکروکنترلر ایجاد نمایید. 4
- اطلاعات کلی و محاسبات 4
- پیکربندی پروژه در STM32CubeMX 6
- پیداسازی برنامه در Keil 9
- شبیه سازی در Proteus 11
- 2- با فعال کردن ماژول های تایمر، وقفه و ADC در میکروکنترلر، تمرین زیر را انجام دهید: یک پتانسیومتر روی پایه ADC1_IN1 قرار دهید و با چرخاندن آن، Duty Cycle سیگنال PWM تولید شده در تمرین ۱ را از 0 تا ۱۰۰٪ تغییر دهید. 13
- اطلاعات کلی و محاسبات 13
- پیکربندی پروژه در STM32CubeMX 15
- پیداسازی برنامه در Keil 18
- شبیه سازی در Proteus 22
- با فعال کردن ماژولهای تایمر، وقفه و ADC در میکروکنترلر، تمرین زیر را انجام دهید: یک سیگنال PWM با Duty Cycle برابر با ۵۰٪ ایجاد نمایید. یک پتانسیومتر روی پایه ADC1_IN1 قرار دهید و با چرخاندن آن، فرکانس سیگنال PWM، را از 0 Hz تا 1 kHz تغییر دهید. تغییرات فرکانس با تغییر موقعیت پتانسیومتر بایستی بصورت کاملاً یکنواخت صورت گیرد و نباید هیچ پرشی در سیگنال تولیدی با اندکی دوران در موقعیت پتانسیومتر، رخ ندهد. مقدار Duty Cycle بایستی کلاً در ۵۰٪ ثابت بماند. 24
- اطلاعات کلی و محاسبات 24
- پیکربندی پروژه در STM32CubeMX 27
- پیداسازی برنامه در Keil 28
- شبیه سازی در Proteus 31
- با فعال کردن ماژولهای تایمر، وقفه و ADC در میکروکنترلر، تمرین زیر را انجام دهید: یک سیگنال PWM ایجاد نمایید بطوریکه فرکانس آن در بازه 0Hz تا 1kHz و Duty Cycle آن در بازه ۱۰۰٪ قابل تغییر باشد. ۲ عدد پتانسیومتر روی پایهها ADC1_IN1 و ADC1_IN2 قرار دهید و با چرخاندن یکی از آنها، فرکانس را در بازه فوق تغییر دهید و با چرخاندن پتانسیومتر دیگر، Duty Cycle را در بازه 0 تا ۱۰۰٪ تغییر دهید. تغییر Duty Cycle و فرکانس با تغییر موقعیت پتانسیومترها بایستی بصورت کاملاً یکنواخت صورت گیرد و نباید هیچ پرشی در مشخصات سیگنال تولیدی با اندکی دوران در موقعیت پتانسیومترها، رخ دهد. تغییر یک کمیت مانند Duty Cycle، نباید باعث تغییر پارامتر دیگر مانند فرکانس شود (وبالعکس). 34
- اطلاعات کلی و محاسبات 34
- پیکربندی پروژه در STM32CubeMX 36
- پیداسازی برنامه در Keil 38

- 42 شبیه‌سازی در Proteus
- تمرین فوق را به جای فعال کردن مازول ADC، با فعال کردن مازول UART و ارسال داده از نرمافزار Matlab، انجام دهید. دقت داشته باشید که اعداد ارسالی از Matlab، برای تغییر Duty Cycle، بایستی بصورت ۱۲ بیتی (بازه 0 تا 4095) باشد همچنین اعداد ارسالی از Matlab، برای تغییر فرکانس بایستی در بازه 0 تا ۱۰۰۰ باشد (قابلیت تغییر فرکانس با دقت 1Hz). در این تمرین، بایستی میکروکنترلر متوجه باشد که کدام عدد ارسالی، برای تغییر فرکانس است و کدام عدد ارسالی برای تغییر Duty Cycle می باشد. 44
- 44 کلیات، محاسبات و منطق برنامه
- 45 تنظیمات STM32CubeMX

فضای توسعه‌ی مورد استفاده با توجه به عدم دسترسی به مخازن نرم افزار بین المللی

1. برای STM32 CubeMX پک F1 نسخه ی V1.8.0

2. برای Keil پک Keil.STM32F1xx_DFP.2.4.0

3. برای Keil پک ARM.CMSIS.4.5.0 متعلق به سال 2015 (آخرین نسخه ای که در نت ملی توانستم

پیدا کنم. الان سال 2026 می باشد یعنی چیزی حدود 11 سال پیش)

4. برای ارتباط سریال با میکرو کنترلر از virtual serial driver pro استفاده شده است که از لینک زیر
دانلود شده است.

<https://p30download.ir/fa/entry/65323/virtual-serial-port-driver>

1- با فعال کردن مازولهای تایمر و وقفه در میکروکنترلر، یک سیگنال PWM با فرکانس 50 Hz و Duty Cycle برابر با ۵۰٪ در یکی از پایه های میکروکنترلر ایجاد نمایید.

اطلاعات کلی و محاسبات

هدف آزمایش

هدف این آزمایش، تولید یک سیگنال PWM با فرکانس 50 هرتز و Duty Cycle برابر با 50 درصد در میکروکنترلر STM32F103C8 است. برخلاف روش های معمول که از واحد سخت افزاری PWM تایمر استفاده می شود، در این پروژه تولید سیگنال به صورت نرم افزاری و تنها با استفاده از وقفه های تایمر (Timer Interrupt) انجام می شود.

در این روش، تایمر تنها نقش ایجاد وقفه را بر عهده دارد و تغییر وضعیت پایه خروجی توسط برنامه انجام می شود. به همین دلیل آشنایی مناسبی با نحوه عملکرد تایمرها، وقفه ها و رجیسترهای داخلی میکروکنترلر ایجاد می شود.

در این پروژه از موارد زیر استفاده شده است:

- میکروکنترلر STM32F103C8
- کلاک داخلی HSI با فرکانس 8 MHz
- تایمر TIM1
- پایه PA1 به عنوان خروجی
- وقفه Update Timer
- وقفه Output Compare Channel 1

مبانی تئوری

تایمرهای STM32 از یک شمارنده داخلی استفاده می کنند که از صفر شروع شده و تا مقدار ARR افزایش پیدا می کند. پس از رسیدن به مقدار ARR دوباره صفر شده و یک وقفه Update تولید می شود.

همچنین اگر مقدار شمارنده با مقدار CCR1 برابر شود، وقفه Output Compare ایجاد می شود.

در این پروژه:

- هنگام وقوع وقفه Update پایه PA1 برابر یک می شود.
- هنگام وقوع وقفه Compare پایه PA1 صفر می شود.

در نتیجه در هر سیکل تایمر، نیمی از زمان خروجی HIGH و نیم دیگر LOW خواهد بود که همان PWM با Duty Cycle پنجاه درصد است.

محاسبات تایمر

فرکانس تایمر از رابطه زیر محاسبه می‌شود:

$$f_{Update} = \frac{f_{CLK}}{(PSC + 1)(ARR + 1)}$$

که در آن:

• فرکانس کلاک :

$$f_{CLK} = 8000000Hz$$

فرکانس مورد نیاز:

$$50Hz$$

ابتدا Prescaler را برابر 19 انتخاب می‌کنیم.

بنابراین:

$$50 = \frac{8000000}{20 \times (ARR + 1)}$$

در نتیجه:

$$ARR + 1 = 8000$$

و خواهیم داشت:

$$ARR = 7999$$

محاسبه Duty Cycle

برای Duty Cycle پنجاه درصد، نقطه تغییر وضعیت خروجی باید در وسط دوره قرار گیرد.

بنابراین:

$$CCR1 = \frac{ARR + 1}{2}$$

در نتیجه:

$$CCR1 = 4000$$

مشخصات نهایی تایمر

مقدار	پارامتر
Clock 8 MHz	
Prescaler 19	
ARR 7999	
CCR1 4000	
فرکانس خروجی 50Hz	
Duty Cycle 50%	

نحوه عملکرد کلی مدار

در ابتدای هر سیکل تایمر، وقفه Update اجرا شده و پایه PA1 برابر یک می‌شود. شمارنده تایمر شروع به افزایش می‌کند. زمانی که شمارنده به مقدار 4000 برسد، وقفه Output Compare اجرا شده و پایه خروجی صفر می‌شود. سپس شمارنده تا مقدار 7999 ادامه پیدا کرده و دوباره سرریز می‌شود. در سرریز بعدی مجدداً پایه خروجی یک شده و این فرآیند به صورت پیوسته تکرار می‌شود. بنابراین شکل موج خروجی یک موج مربعی با فرکانس 50 Hz و Duty Cycle برابر 50 درصد خواهد بود.

پیکربندی پروژه در STM32CubeMX

پس از ایجاد پروژه جدید در نرم‌افزار STM32CubeMX، میکروکنترلر STM32F103C8Tx انتخاب شد.

تنظیمات SYS

در بخش SYS گزینه Debug روی Serial Wire قرار گرفت تا امکان برنامه‌ریزی و دیباگ توسط ST-Link فراهم شود.

تنظیمات RCC

در قسمت RCC از کلاک داخلی HSI استفاده شد و اسیلاتور خارجی (HSE) غیرفعال باقی ماند.

فرکانس HSI برابر 8 MHz است که برای تولید سیگنال مورد نظر کافی می‌باشد.

تنظیم پایه خروجی

پایه PA0 به صورت GPIO Output تنظیم شد.

این پایه وظیفه تولید موج PWM را بر عهده دارد.

خروجی این پایه در محیط Proteus به اسیلوسکوپ متصل خواهد شد.

تنظیم تایمر TIM1

در قسمت Timers ، تایمر TIM1 انتخاب شد.

تنظیمات به صورت زیر انجام شد:

Clock Source:

Internal Clock

Channel 1:

Output Compare No Output

انتخاب این حالت باعث می‌شود تایمر تنها وقفه Compare ایجاد کند و هیچ خروجی سخت‌افزاری PWM روی پایه فعال نشود.

تنظیمات پارامترهای تایمر

در بخش Parameter Settings مقادیر زیر وارد شدند:

Prescaler = 19

Counter Period = 7999

Counter Mode = Up

Clock Division = No Division

Auto Reload Preload = Disable

این تنظیمات باعث ایجاد یک وقفه Update هر 20 میلی‌ثانیه می‌شود.

تنظیم Output Compare

در قسمت Output Compare مقدار Pulse برابر با 4000 وارد شد.

این مقدار همان CCR1 است.

به محض اینکه شمارنده به مقدار 4000 برسد، وقفه Compare اجرا خواهد شد.

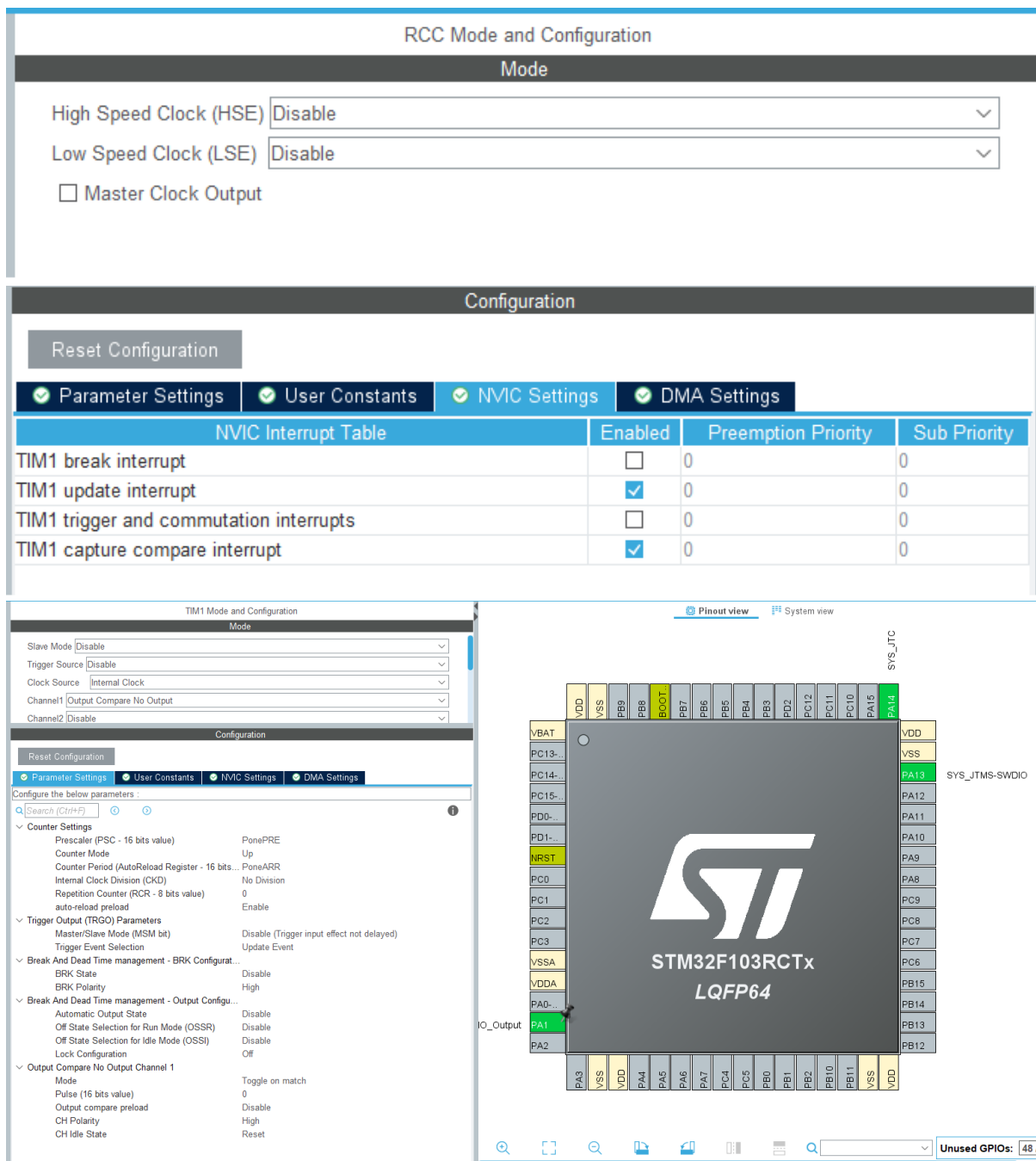
فعال کردن وقفه

در بخش NVIC گزینه

TIM2 Global Interrupt

فعال شد.

بدون فعال بودن این گزینه هیچ Callback اجرا نخواهد شد.



تولید کد

در قسمت Project Manager نرم افزار Keil MDK-ARM به عنوان IDE انتخاب شد.

در پایان گزینه Generate Code فشرده شد تا فایل های پروژه ایجاد شوند.

پیاده سازی برنامه در Keil

پس از تولید پروژه، فایل main.c باز شده و تنها بخش های User Code ویرایش شدند.

ابتدا متغیر های را که در cubemx برای PRE و ARR انتخاب کردیم مقدار دهی میکنیم

```
/* USER CODE BEGIN PV */
uint16_t PonePRE = 19, PoneARR = 7999 ,PonePULSE = 4000;
/* USER CODE END PV */
```

توابع کالبدی برای ست و ریست کردن پین 1 با توجه به اینترایت ها

```
/* USER CODE BEGIN 0 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    if(TIM1 -> CCR1){
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_SET);
    }
}
void HAL_TIM_OC_DelayElapsedCallback(TIM_HandleTypeDef *htim){
    if(htim -> Channel == HAL_TIM_ACTIVE_CHANNEL_1){
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_RESET);
    }
}
/* USER CODE END 0 */
```

شروع کردن تایمر و اینترایت آن و output compare اینترایت تایمر

```
/* USER CODE BEGIN 2 */

HAL_TIM_Base_Start_IT(&htim1);
HAL_TIM_OC_Start_IT(&htim1, TIM_CHANNEL_1);

/* USER CODE END 2 */
```

ست کردن اندازه پالس برای ایجاد 50% duty cycle

```
/* USER CODE BEGIN WHILE */
while (1)
{
    TIM1->CCR1 = PonePULSE;
}
/* USER CODE END WHILE */
```

روند اجرای برنامه

پس از روشن شدن میکروکنترلر:

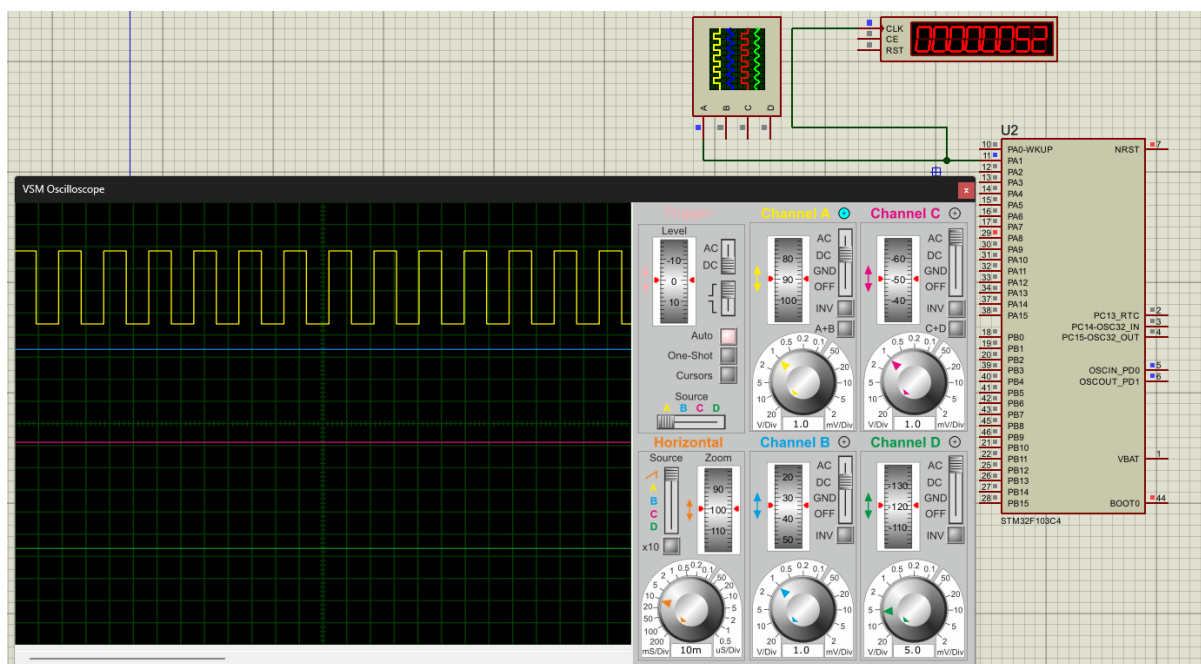
1. تایمر شروع به شمارش می کند .
2. در ابتدای سیکل خروجی HIGH می شود .
3. در مقدار 4000 خروجی LOW می شود .

4. شمارنده تا 7999 ادامه پیدا می‌کند.

5. تایمر سرریز شده و دوباره خروجی HIGH می‌شود.

این فرآیند به طور مداوم تکرار می‌شود و موج PWM تولید می‌شود.

شبیه‌سازی در Proteus



ساخت مدار

در محیط Proteus قطعات زیر انتخاب شدند:

- 4STM32F103C
- Oscilloscope
- منبع تغذیه 3.3 ولت
- زمین (Ground)

اتصال پایه‌ها

پایه PA1 میکروکنترلر به کانال A اسیلوسکوپ متصل شد.
پایه‌های تغذیه نیز مطابق دیتاشیت به VCC و GND متصل شدند.

بارگذاری فایل برنامه

پس از Build پروژه در Keil فایل

Project.hex

تولید شد.

در قسمت Program File میکروکنترلر همین فایل انتخاب گردید.

همچنین فرکانس کریستال روی

8MHz

قرار داده شد تا با تنظیمات CubeMX مطابقت داشته باشد.

اجرای شبیه‌سازی

با اجرای Simulation ، اسیلوسکوپ شکل موج مربعی تولید شده روی پایه PA0 را نمایش داد.

با تنظیم Time/Div روی حدود **5ms/div**، یک دوره کامل سیگنال به وضوح مشاهده شد.

تحلیل نتایج

بررسی شکل موج خروجی نشان داد که:

- دوره تناوب سیگنال برابر **20 میلی‌ثانیه** است که معادل **50 هرتز** می‌باشد .
- زمان HIGH تقریباً **10 میلی‌ثانیه** است .
- زمان LOW نیز حدود **10 میلی‌ثانیه** است .
- بنابراین **Duty Cycle** برابر **50 درصد** به دست آمده است .

از آنجا که تغییر وضعیت پایه خروجی صرفاً توسط وقفه‌های **Update** و **Output Compare** انجام می‌شود و هیچ‌یک از مدهای سخت‌افزاری PWM تایمر مورد استفاده قرار نگرفته‌اند، هدف اصلی این آزمایش با موفقیت محقق شده است. همچنین نتایج شبیه‌سازی در Proteus با محاسبات اولیه کاملاً مطابقت داشته و عملکرد صحیح برنامه را تأیید می‌کند.

ویدیو ی شبیه سازی در پوشه سوال موجود می باشد.

2- با فعال کردن مازول های تایمر، وقفه و ADC در میکروکنترلر، تمرین زیر را انجام دهید: یک پتانسیومتر روی پایه ADC1_IN1 قرار دهید و با چرخاندن آن، Duty Cycle سیگنال PWM تولید شده در تمرین ۱ را از 0 تا ۱۰۰٪ تغییر دهید.

اطلاعات کلی و محاسبات

هدف آزمایش

هدف این آزمایش، کنترل **Duty Cycle** یک سیگنال PWM با استفاده از مقدار ولتاژ ورودی یک پتانسیومتر است. در این پروژه، ولتاژ خروجی پتانسیومتر توسط مبدل آنالوگ به دیجیتال (ADC) خوانده شده و مقدار حاصل به صورت لحظه‌ای به **Duty Cycle** سیگنال PWM تبدیل می‌شود. در نتیجه، با چرخاندن پتانسیومتر، نسبت زمان روشن بودن سیگنال از 0 درصد تا 100 درصد تغییر خواهد کرد، در حالی که فرکانس سیگنال همواره برابر 50 هرتز باقی می‌ماند.

در این تمرین همچنان تولید سیگنال PWM به وسیله وقفه‌های تایمر انجام می‌شود و از واحد سخت‌افزاری PWM استفاده نشده است.

معرفی اجزای مورد استفاده

در این پروژه از منابع سخت‌افزاری زیر استفاده شده است:

- میکروکنترلر STM32F103C8
- کلاک داخلی HSI با فرکانس 8 MHz
- تایمر TIM1
- مبدل آنالوگ به دیجیتال ADC1
- کانال ADC1_IN1
- پتانسیومتر جهت تولید ولتاژ متغیر
- پایه PA1 به عنوان ورودی آنالوگ
- پایه PA0 به عنوان خروجی PWM

لازم به ذکر است که در STM32 پایه PA1 به صورت سخت‌افزاری به کانال ADC1_IN1 متصل است. به همین دلیل امکان استفاده همزمان از این پایه برای خروجی PWM وجود ندارد و پایه خروجی PWM از PA1 به PA0 منتقل شده است.

نحوه عملکرد سیستم

ابتدا ولتاژ خروجی پتانسیومتر وارد پایه PA1 می‌شود.

این ADC ولتاژ را نمونه‌برداری کرده و آن را به یک عدد دیجیتال تبدیل می‌کند.

برنامه مقدار دیجیتال را دریافت کرده و آن را به مقدار مناسب رجیستر CCR1 تبدیل می‌کند.

در نهایت مقدار جدید CCR1 تعیین می‌کند که خروجی PWM چه مدت در وضعیت HIGH باقی بماند.

در نتیجه:

- با کم شدن ولتاژ پتانسیومتر Duty Cycle کاهش می‌یابد .
- با افزایش ولتاژ پتانسیومتر Duty Cycle افزایش پیدا می‌کند .
- فرکانس خروجی در تمام حالات ثابت باقی می‌ماند .

محاسبات ADC

مبدل ADC در STM32F103 دارای دقت 12 بیت است.

بنابراین خروجی آن در بازه زیر قرار می‌گیرد:

$$0 \leq ADC \leq 4095$$

در این پروژه مقدار ARR برابر است با:

$$ARR = 7999$$

از آنجایی که CCR باید مقداری بین صفر تا ARR داشته باشد، لازم است مقدار ADC به این بازه تبدیل شود.

برای این منظور از رابطه زیر استفاده می‌شود:

$$CCR1 = \frac{ADC \times 7999}{4095}$$

این رابطه یک نگاشت خطی بین ولتاژ ورودی و Duty Cycle ایجاد می‌کند.

بررسی چند نمونه

اگر خروجی ADC صفر باشد:

$$CCR1 = 0$$

در نتیجه خروجی همیشه LOW خواهد بود.

اگر ADC برابر 2048 باشد:

$$CCR1 = \frac{2048 \times 7999}{4095} \approx 4000$$

بنابراین Duty Cycle تقریباً برابر 50 درصد خواهد شد.

اگر مقدار ADC برابر 4095 باشد:

$$CCR1 = 7999$$

در این حالت خروجی تقریباً همیشه HIGH باقی می‌ماند و Duty Cycle برابر 100 درصد خواهد بود.

جدول تبدیل ADC به Duty Cycle

مقدار ADC مقدار CCR1 Duty Cycle

0	0	0%
1024	≈2000	25%
2048	≈4000	50%
3072	≈6000	75%
4095	7999	100%

پیکربندی پروژه در STM32CubeMX

ابتدا پروژه جدیدی برای میکروکنترلر STM32F103C8Tx ایجاد شد.

تنظیمات پایه‌های ورودی و خروجی

از آنجا که پایه PA1 باید به عنوان ورودی ADC استفاده شود، این پایه روی حالت

ADC1_IN1

قرار گرفت.

سپس پایه PA0 به عنوان

GPIO Output

انتخاب شد تا سیگنال PWM روی این پایه تولید شود.

پی‌کر بندی ADC

در بخش Analog ، واحد ADC1 فعال شد.

تنها کانال

ADC1_IN1

انتخاب گردید.

سایر تنظیمات روی مقادیر پیش فرض باقی ماندند، زیرا تنها به یک تبدیل تکی (Single Conversion) نیاز بود.

تنظیم تایمر TIM1

در بخش Timers ، تایمر TIM1 فعال شد.

تنظیمات آن مشابه تمرین قبل انجام شد.

پارامترهای اصلی عبارت‌اند از:

- Clock Source = Internal Clock
- Counter Mode = Up
- Prescaler = 19
- Counter Period = 7999

تنظیم Output Compare

برای کانال اول تایمر، حالت

Output Compare No Output

انتخاب شد تا تنها وقفه Compare تولید شود و پایه خروجی مستقیماً توسط سخت افزار کنترل نشود.

فعال کردن وقفه‌ها

در قسمت NVIC ، وقفه TIM1 فعال شد تا Callback های مربوط به Update و Output Compare اجرا شوند .

TIM1 Mode and Configuration

Mode

Slave Mode: Disable

Trigger Source: Disable

Clock Source: Internal Clock

Channel1: Output Compare No Output

Channel2: Disable

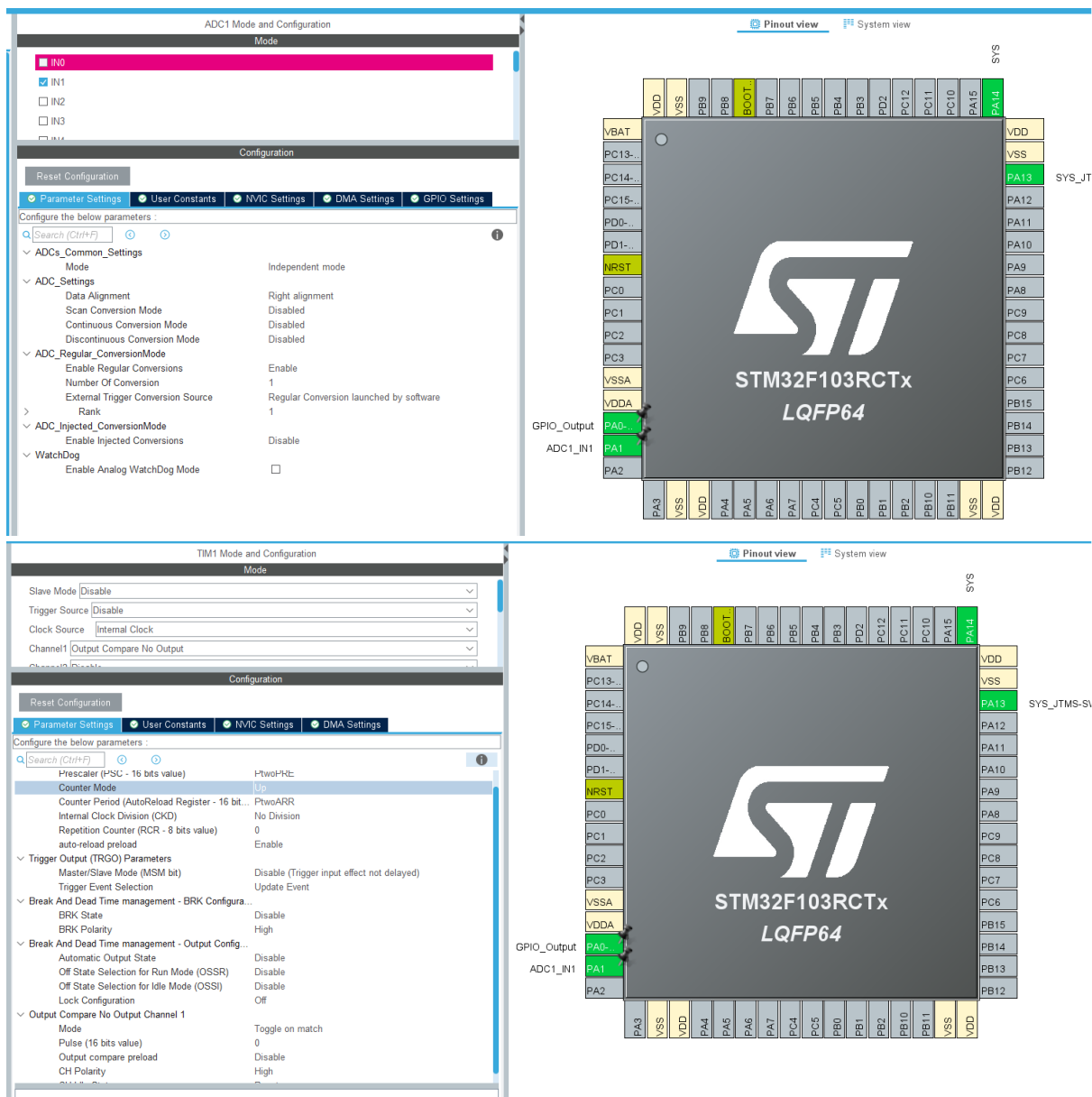
Configuration

Reset Configuration

Parameter Settings User Constants NVIC Settings DMA Settings

NVIC Interrupt Table	Enabled	Presmption Priority	Sub Priority
TIM1 break interrupt	☐	0	0
TIM1 update interrupt	☒	0	0
TIM1 trigger and commutation interrupts	☐	0	0
TIM1 capture compare interrupt	☒	0	0

Pinout view System view



تولید پروژه

در نهایت IDE نرم افزار روی Keil MDK-ARM قرار گرفت و پروژه توسط گزینه **Generate Code** تولید شد.

پیاپی سازی برنامه در Keil

پس از تولید پروژه توسط CubeMX، بخش های مورد نیاز در فایل **main.c** تکمیل شدند.

تعریف متغیرها

```
uint16_t PtwoPRE = 19;
uint16_t PtwoARR = 7999;
uint32_t adc_value = 0;
```

توضیح کد

دو متغیر اول مقادیر Prescaler و ARR را نگهداری می‌کنند تا در ابتدای برنامه داخل رجیسترهای تایمر قرار گیرند. متغیر **adc_value** نیز مقدار تبدیل شده توسط ADC را ذخیره می‌کند تا در محاسبه Duty Cycle مورد استفاده قرار گیرد.

مقداردهی اولیه تایمر

```
TIM1->CCR1 = 0;
```

توضیح کد

ابتدا مقدار Prescaler و ARR داخل رجیسترهای تایمر قرار می‌گیرند. سپس مقدار CCR1 برابر صفر قرار داده می‌شود تا در ابتدای اجرای برنامه Duty Cycle برابر صفر درصد باشد و خروجی خاموش بماند.

راه اندازی وقفه‌های تایمر

```
HAL_TIM_Base_Start_IT(&htim1);
```

```
HAL_TIM_OC_Start_IT(&htim1, TIM_CHANNEL_1);
```

توضیح کد

تابع اول وقفه سرریز تایمر را فعال می‌کند. تابع دوم وقفه Output Compare کانال اول را فعال می‌کند. در نتیجه در هر سیکل دو وقفه اجرا خواهند شد.

تابع وقفه Update

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if(TIM1->CCR1 > 0)
    {
        HAL_GPIO_WritePin(GPIOA,
                           GPIO_PIN_0,
                           GPIO_PIN_SET);
    }
}
```

توضیح کد

این تابع هنگام سرریز تایمر اجرا می‌شود.
ابتدا بررسی می‌شود که مقدار CCR1 بزرگتر از صفر باشد.
اگر Duty Cycle برابر صفر باشد خروجی روشن نمی‌شود.
در غیر این صورت پایه PA0 در وضعیت HIGH قرار می‌گیرد و ابتدای سیکل PWM آغاز می‌شود.

تابع وقفه Output Compare

```
void HAL_TIM_OC_DelayElapsedCallback
(
TIM_HandleTypeDef *htim
)
{
    if(htim->Channel==
        HAL_TIM_ACTIVE_CHANNEL_1)
    {
        HAL_GPIO_WritePin(GPIOA,
            GPIO_PIN_0,
            GPIO_PIN_RESET);
    }
}
```

توضیح کد

زمانی که شمارنده تایمر به مقدار CCR1 برسد این تابع اجرا می‌شود.
در این لحظه پایه PA0 خاموش شده و وارد وضعیت LOW می‌شود.
هرچه مقدار CCR1 بزرگتر باشد، مدت زمان HIGH بیشتر خواهد بود و در نتیجه Duty Cycle افزایش می‌یابد.

خواندن مقدار ADC

```
HAL_ADC_Start(&hadc1);
```

توضیح کد

این دستور فرآیند نمونه‌برداری و تبدیل آنالوگ به دیجیتال را آغاز می‌کند.

انتظار برای پایان تبدیل

```
HAL_ADC_PollForConversion
(
```

```
&hadc1,  
10  
);
```

توضیح کد

پردازنده تا حداکثر 10 میلی ثانیه منتظر پایان تبدیل ADC می ماند.
در صورت موفق بودن تبدیل، مقدار دیجیتال آماده خواندن خواهد بود.

دریافت مقدار ADC

```
adc_value =  
HAL_ADC_GetValue(&hadc1);
```

توضیح کد

این دستور مقدار 12 بیتی خروجی ADC را دریافت کرده و در متغیر **adc_value** ذخیره می کند.
این مقدار عددی بین صفر تا 4095 خواهد بود.

محاسبه Duty Cycle

```
TIM1->CCR1 =  
(adc_value*PtwoARR)  
/4095;
```

توضیح کد

این مهمترین بخش برنامه است.
مقدار خوانده شده از ADC به صورت خطی به بازه صفر تا ARR تبدیل می شود.
بنابراین:

- اگر ADC کوچک باشد، CCR نیز کوچک خواهد بود .
 - اگر ADC بزرگ باشد، CCR نیز بزرگ می شود .
- در نتیجه مدت زمان روشن بودن خروجی مستقیماً با ولتاژ ورودی متناسب خواهد بود.

تأخیر برنامه

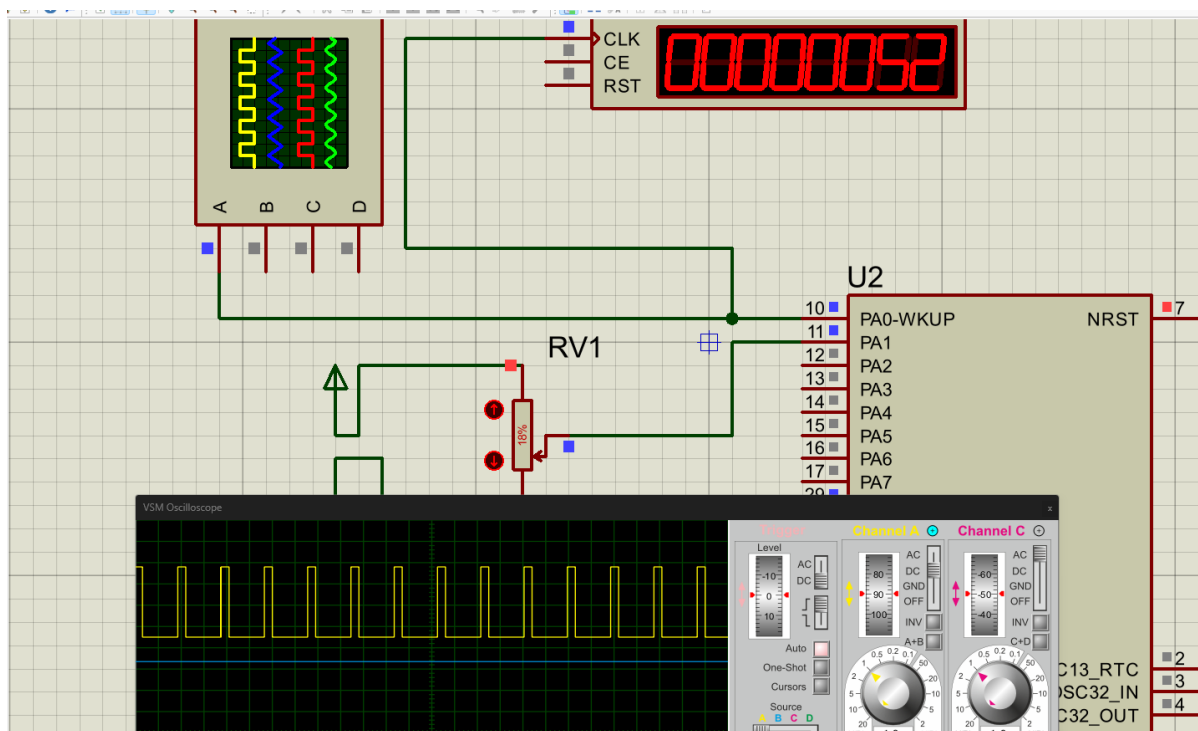
```
HAL_Delay(10);
```

توضیح کد

این تأخیر کوتاه باعث می‌شود تعداد نمونه‌برداری‌ها بیش از حد زیاد نشده و شبیه‌سازی در Proteus با پایداری بیشتری اجرا شود.

شبیه‌سازی در Proteus

برای بررسی عملکرد مدار، شبیه‌سازی در نرم‌افزار Proteus انجام شد.



قطعات مورد استفاده

مدار شامل قطعات زیر بود:

- STM32F103C8
- پتانسیومتر POT-HG
- Oscilloscope
- منبع تغذیه 3.3 V
- Ground

نحوه اتصال مدار

دو سر پتانسیومتر به ولتاژ 3.3 ولت و زمین متصل شدند.

پایه میانی (Wiper) مستقیماً به پایه PA1 وصل شد تا ولتاژ متغیر وارد ADC شود.

پایه PA0 نیز به کانال A اسیلوسکوپ متصل گردید تا شکل موج PWM نمایش داده شود.

بارگذاری فایل برنامه

پس از کامپایل پروژه در Keil ، فایل HEX تولید شد.

این فایل در قسمت **Program File** میکروکنترلر در Proteus بارگذاری گردید.

فرکانس میکروکنترلر نیز روی 8MHz تنظیم شد.

نتایج شبیه‌سازی

با اجرای شبیه‌سازی مشاهده شد که با تغییر وضعیت پتانسیومتر، عرض پالس خروجی به صورت پیوسته تغییر می‌کند.

هنگامی که پتانسیومتر در کمترین مقدار قرار داشت، خروجی تقریباً همیشه LOW بود و Duty Cycle نزدیک به صفر درصد مشاهده شد.

با قرار گرفتن پتانسیومتر در موقعیت میانی، زمان HIGH و LOW تقریباً برابر شد و Duty Cycle حدود 50 درصد به دست آمد.

در بیشترین مقدار ولتاژ نیز خروجی تقریباً در تمام سیکل در وضعیت HIGH باقی ماند و Duty Cycle به حدود 100 درصد رسید.

همچنین اندازه‌گیری‌ها نشان داد که فرکانس سیگنال در تمام حالات برابر 50 هرتز باقی مانده است و تنها عرض پالس تغییر می‌کند. این موضوع نشان می‌دهد که طراحی سیستم به درستی انجام شده و هدف اصلی آزمایش، یعنی کنترل Duty Cycle سیگنال PWM با استفاده از مقدار ولتاژ ورودی ADC ، با موفقیت محقق شده است.

با فعال کردن ماژولهای تایمر، وقفه و ADC در میکروکنترلر، تمرین زیر را انجام دهید: یک سیگنال PWM با Duty Cycle برابر با ۵۰٪ ایجاد نمایید. یک پتانسیومتر روی پایه ADC1_IN1 قرار دهید و با چرخاندن آن، فرکانس سیگنال PWM، را از ۰ Hz تا ۱ kHz تغییر دهید. تغییرات فرکانس با تغییر موقعیت پتانسیومتر بایستی بصورت کاملاً یکنواخت صورت گیرد و نباید هیچ پرشی در سیگنال تولیدی با اندکی دوران در موقعیت پتانسیومتر، رخ ندهد. مقدار Duty Cycle بایستی کلاً در ۵۰٪ ثابت بماند.

اطلاعات کلی و محاسبات

هدف آزمایش

هدف این آزمایش، تولید یک سیگنال PWM با Duty Cycle ثابت ۵۰ درصد و فرکانس قابل تنظیم از ۰ تا ۱۰۰۰ هرتز با استفاده از ولتاژ خروجی یک پتانسیومتر است. مقدار ولتاژ پتانسیومتر توسط واحد مبدل آنالوگ به دیجیتال (ADC) اندازه‌گیری شده و بر اساس آن فرکانس خروجی تایمر تغییر می‌کند. در این پروژه تغییر فرکانس باید کاملاً پیوسته و بدون پرش (Glitch-Free) انجام شود، به گونه‌ای که تغییر مقدار فرکانس باعث ایجاد اعوجاج یا تغییر ناگهانی در شکل موج خروجی نشود.

برای تحقق این هدف از قابلیت **Auto-Reload Preload** در تایمر استفاده می‌شود. این قابلیت باعث می‌شود مقدار جدید رجیستر ARR ابتدا در یک بافر ذخیره شده و تنها پس از پایان سیکل جاری تایمر اعمال شود. در نتیجه تغییر فرکانس بدون ایجاد ناپایداری در خروجی انجام خواهد شد.

معرفی اجزای مورد استفاده

در این پروژه از تجهیزات و واحدهای داخلی زیر استفاده شده است:

- میکروکنترلر STM32F103C8
- کلاک داخلی HSI با فرکانس ۸ MHz
- تایمر TIM1
- مبدل آنالوگ به دیجیتال ADC1
- کانال ADC1_IN1
- پتانسیومتر
- پایه PA1 به عنوان ورودی ADC

- پایه PA0 به عنوان خروجی PWM

در این تمرین نیز پایه PA1 تنها به عنوان ورودی آنالوگ مورد استفاده قرار گرفته و خروجی PWM روی پایه PA0 تولید می‌شود.

نحوه عملکرد سیستم

ابتدا ولتاژ خروجی پتانسیومتر توسط ADC نمونه‌برداری می‌شود. برنامه مقدار دیجیتال حاصل را دریافت کرده و آن را به فرکانس متناظر در بازه صفر تا 1000 هرتز تبدیل می‌کند. سپس مقدار جدید رجیستر ARR محاسبه شده و در تایمر قرار می‌گیرد. به دلیل فعال بودن قابلیت Auto-Reload Preload، مقدار جدید ARR بلافاصله اعمال نشده و تنها پس از پایان سیکل جاری وارد رجیستر اصلی تایمر می‌شود. بنابراین تغییر فرکانس کاملاً نرم و بدون پرش انجام خواهد شد. برای ثابت ماندن Duty Cycle روی مقدار 50 درصد نیز مقدار CCR1 همواره برابر نصف ARR تنظیم می‌شود.

محاسبات تایمر

برای ساده‌تر شدن محاسبات، مقدار Prescaler برابر 79 انتخاب شده است.

در نتیجه فرکانس شمارش تایمر برابر خواهد بود با:

$$f_{TIM} = \frac{8000000}{79 + 1} = 100000Hz$$

بنابراین تایمر با فرکانس 100 کیلوهرتز شمارش می‌کند.

محاسبه فرکانس خروجی

مقدار ADC در بازه زیر قرار دارد:

$$0 \leq ADC \leq 4095$$

فرکانس خروجی نیز باید بین صفر تا 1000 هرتز تغییر کند.

بنابراین رابطه تبدیل ADC به فرکانس برابر است با:

$$f = \frac{ADC \times 1000}{4095}$$

محاسبه ARR

فرکانس تایمر از رابطه زیر به دست می‌آید:

$$f = \frac{100000}{ARR + 1}$$

بنابراین:

$$ARR = \frac{100000}{f} - 1$$

با جایگذاری رابطه فرکانس خواهیم داشت:

$$ARR = \frac{409500}{ADC} - 1$$

این رابطه بدون استفاده از محاسبات اعشاری قابل پیاده‌سازی در زبان C است و سرعت اجرای برنامه را افزایش می‌دهد.

محاسبه Duty Cycle

برای ثابت نگه داشتن Duty Cycle روی مقدار 50 درصد، مقدار رجیستر CCR1 همیشه برابر نصف دوره در نظر گرفته می‌شود:

$$CCR1 = \frac{ARR + 1}{2}$$

در نتیجه با تغییر فرکانس، Duty Cycle همواره برابر 50 درصد باقی می‌ماند.

جدول عملکرد سیستم

وضعیت خروجی فرکانس تقریبی مقدار ADC

0	خروجی خاموش	0 Hz
1024	حدود	Duty=50% Hz250
2048	حدود	Duty=50% Hz500
3072	حدود	Duty=50% Hz750
4095		Duty=50% 1000 Hz

پیکربندی پروژه در STM32CubeMX

ابتدا پروژه جدیدی برای میکروکنترلر STM32F103C8Tx ایجاد شد.

تنظیم پایه‌های ورودی و خروجی

پایه PA1 به عنوان

ADC1_IN1

پیکربندی شد تا ولتاژ خروجی پتانسیومتر را دریافت کند.

پایه PA0 نیز به صورت

GPIO Output

تنظیم شد تا سیگنال PWM روی آن تولید شود.

فعال‌سازی ADC

در بخش Analog ، واحد ADC1 فعال گردید و کانال ADC1_IN1 انتخاب شد.

سایر تنظیمات ADC روی مقادیر پیش‌فرض باقی ماندند.

تنظیم تایمر TIM1

در بخش Timers ، تایمر TIM1 فعال شد.

تنظیمات اصلی عبارت بودند از:

• Clock Source = Internal Clock

• Counter Mode = Up

• Prescaler = 79

برای کانال اول نیز حالت

Output Compare No Output

انتخاب شد.

فعال‌سازی Auto-Reload Preload

مهم‌ترین تغییر این تمرین، فعال کردن گزینه

Auto Reload Preload

بود.

در صورت غیرفعال بودن این گزینه، هر بار که مقدار ARR تغییر کند، فرکانس خروجی به صورت ناگهانی تغییر کرده و ممکن است پالس ناقص یا اعوجاج در خروجی مشاهده شود.

با فعال بودن این قابلیت، مقدار جدید ARR تنها پس از پایان سیکل جاری اعمال می‌شود و تغییر فرکانس کاملاً نرم خواهد بود.

فعال کردن وقفه‌ها

در قسمت NVIC ، وقفه TIM1 فعال شد تا Callback های مربوط به Update و Output Compare اجرا شوند.

تولید پروژه

در پایان IDE روی Keil MDK-ARM قرار داده شد و پروژه توسط CubeMX تولید گردید.

پیاده‌سازی برنامه در Keil

پس از تولید پروژه، بخش‌های موردنیاز در فایل main.c تکمیل شدند.

تعریف متغیرها

```
uint16_t PthreePRE = 79;  
uint32_t adc_value = 0;  
uint32_t PthreeARR = 0;
```

توضیح کد

متغیر **PthreePRE** مقدار Prescaler را نگهداری می‌کند.

متغیر **adc_value** مقدار خوانده شده از ADC را ذخیره می‌کند.

متغیر **PthreeARR** نیز مقدار جدید رجیستر ARR را نگهداری می‌کند تا پس از محاسبه در تایمر قرار گیرد.

توضیح کد

در این قسمت مقدار Prescaler داخل رجیستر تایمر قرار می‌گیرد تا فرکانس شمارش تایمر برابر 100 کیلوهرتز شود.

خواندن مقدار ADC

```
HAL_ADC_Start(&hadc1);
```

توضیح کد

این دستور تبدیل آنالوگ به دیجیتال را آغاز می‌کند.

```
HAL_ADC_PollForConversion(&hadc1,10);
```

توضیح کد

پردازنده تا پایان عملیات تبدیل منتظر می‌ماند.

```
adc_value = HAL_ADC_GetValue(&hadc1);
```

توضیح کد

مقدار 12 بیتی ADC دریافت شده و در متغیر **adc_value** ذخیره می‌شود.

بررسی حالت صفر هرتز

```
if(adc_value<5)
{
    HAL_TIM_Base_Stop_IT(&htim1);

    HAL_TIM_OC_Stop_IT(&htim1,
        TIM_CHANNEL_1);

    HAL_GPIO_WritePin(GPIOA,
        GPIO_PIN_0,
        GPIO_PIN_RESET);
}
```

توضیح کد

اگر مقدار ADC تقریباً صفر باشد، فرکانس خروجی نیز باید صفر شود.
در این حالت وقفه‌های تایمر متوقف شده و پایه خروجی LOW باقی می‌ماند.
این کار از بروز خطای تقسیم بر صفر در محاسبات جلوگیری می‌کند.

راه‌اندازی مجدد تایمر

```
HAL_TIM_Base_Start_IT(&htim1);
```

```
HAL_TIM_OC_Start_IT(&htim1,  
TIM_CHANNEL_1);
```

توضیح کد

اگر مقدار ADC بزرگتر از صفر باشد، تایمر دوباره فعال شده و تولید سیگنال از سر گرفته می‌شود.

محاسبه مقدار جدید ARR

```
PthreeARR =  
(409500/adc_value)-1;
```

توضیح کد

این دستور مقدار جدید رجیستر ARR را بر اساس مقدار ADC محاسبه می‌کند.

هرچه مقدار ADC بیشتر باشد، مقدار ARR کوچکتر شده و در نتیجه فرکانس خروجی افزایش می‌یابد.

محدود کردن مقدار ARR

```
if(PthreeARR >65535)  
{  
PthreeARR =65535;  
}
```

توضیح کد

از آنجا که رجیستر ARR در تایمر 16 بیتی است، مقدار آن نباید از 65535 بیشتر شود.

این شرط از بروز سرریز جلوگیری می‌کند.

اعمال فرکانس جدید

```
TIM1->ARR= PthreeARR;
```

توضیح کد

مقدار جدید ARR داخل رجیستر تایمر قرار می‌گیرد.

به دلیل فعال بودن قابلیت Auto-Reload Preload ، این مقدار در انتهای سیکل جاری اعمال می‌شود و تغییر فرکانس بدون پرش انجام می‌شود.

ثابت نگه داشتن Duty Cycle

TIM1->CCR1=
(PthreeARR +1)/2;

توضیح کد

برای آنکه نسبت سیکل کاری همواره برابر 50 درصد باقی بماند، مقدار CCR1 همیشه نصف ARR قرار داده می‌شود. در نتیجه با هر تغییر فرکانس، Duty Cycle تغییر نخواهد کرد.

تاخیر برنامه

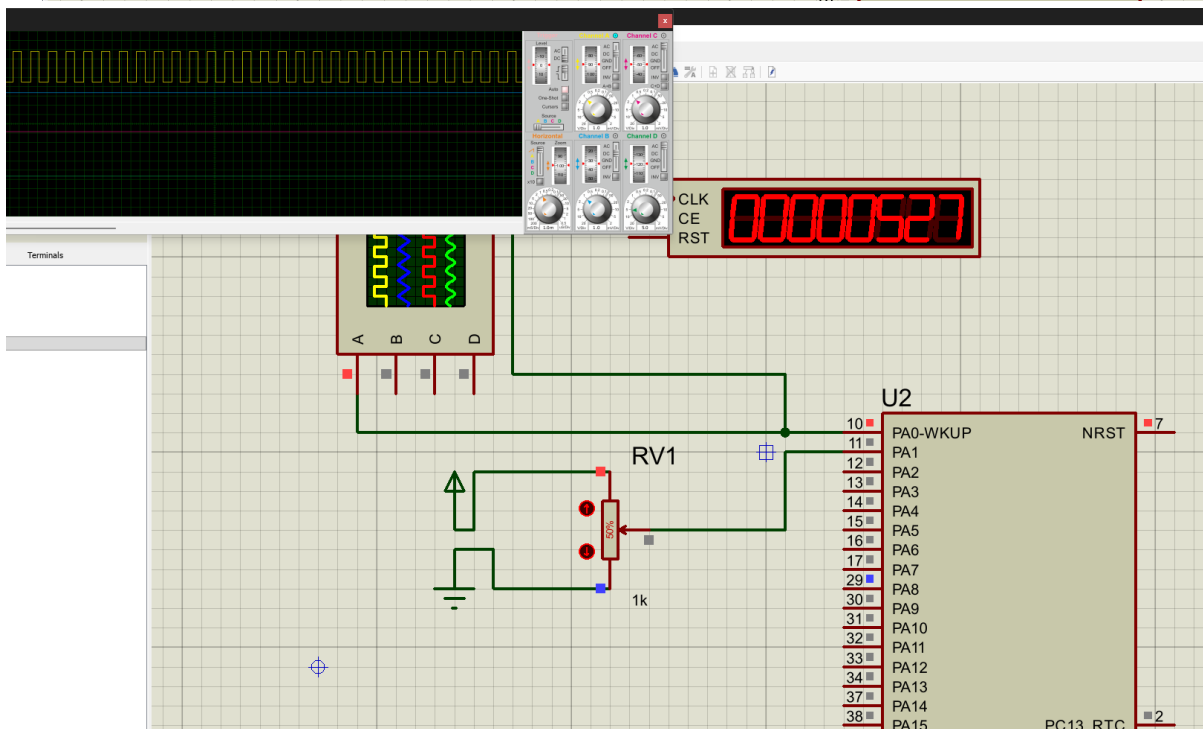
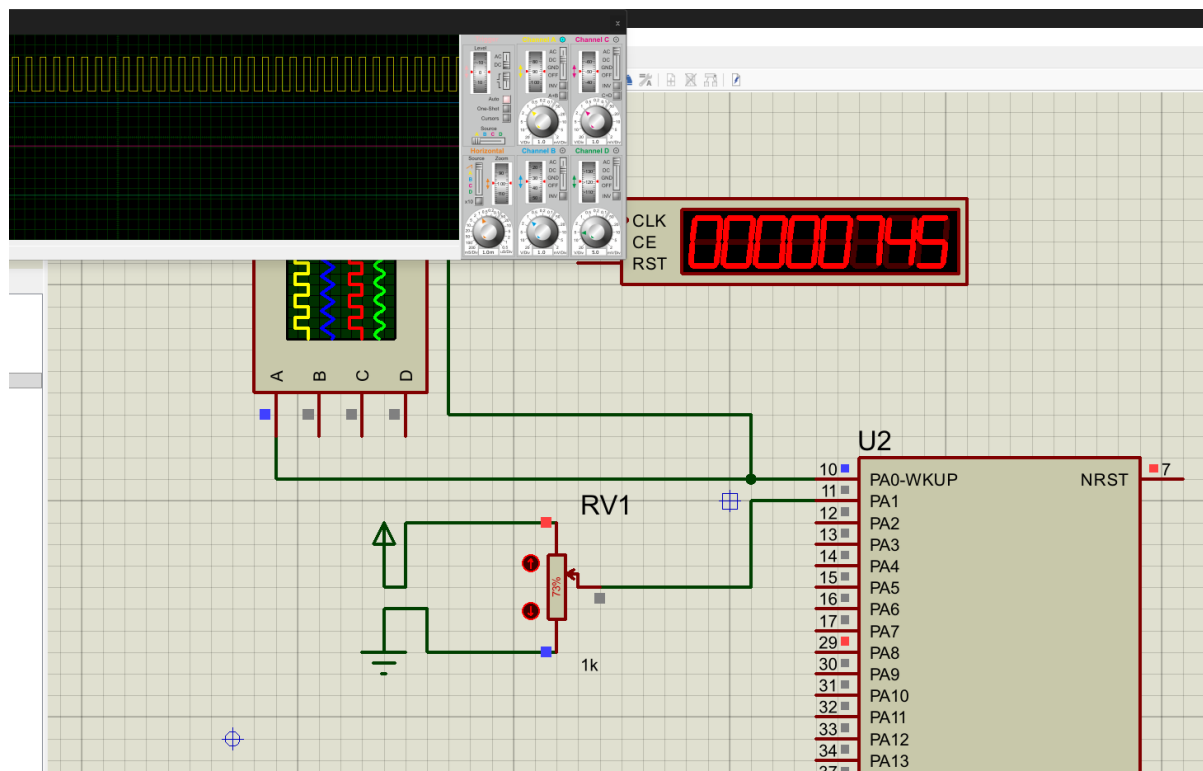
HAL_Delay(10);

توضیح کد

این تاخیر کوتاه باعث کاهش تعداد نمونه‌برداری‌ها و افزایش پایداری شبیه‌سازی در Proteus می‌شود.

شبیه‌سازی در Proteus

برای بررسی عملکرد مدار از نرم‌افزار Proteus استفاده شد.



قطعات مورد استفاده

در مدار از قطعات زیر استفاده شد:

- STM32F103C8
- پتانسیومتر POT-HG

- Oscilloscope
- Frequency Counter
- منبع تغذیه 3.3 ولت
- Ground

نحوه اتصال مدار

دو سر پتانسیومتر به منبع تغذیه و زمین متصل شدند.

پایه میانی پتانسیومتر به پایه PA1 وصل شد تا ولتاژ متغیر وارد ADC گردد.

پایه PA0 نیز به صورت همزمان به اسیلوسکوپ و Frequency Counter متصل شد تا شکل موج و فرکانس خروجی قابل مشاهده باشد.

بارگذاری فایل برنامه

پس از Build پروژه در Keil ، فایل HEX تولید شد.

این فایل در قسمت **Program File** میکروکنترلر در Proteus بارگذاری گردید و فرکانس کاری میکروکنترلر روی 8MHz تنظیم شد.

تحلیل نتایج

با اجرای شبیه‌سازی مشاهده شد که با چرخاندن پتانسیومتر، فرکانس سیگنال خروجی به صورت پیوسته از 0 هرتز تا 1000 هرتز تغییر می‌کند. در کمترین مقدار ولتاژ، خروجی در سطح LOW ثابت باقی مانده و فرکانس صفر هرتز مشاهده می‌شود. با افزایش ولتاژ ورودی، فرکانس به تدریج افزایش یافته و در بیشترین مقدار پتانسیومتر به حدود 1000 هرتز می‌رسد.

همچنین بررسی شکل موج در اسیلوسکوپ نشان داد که نسبت سیکل کاری در تمام محدوده تغییر فرکانس برابر 50 درصد باقی مانده است و به دلیل فعال بودن قابلیت **Auto-Reload Preload**، تغییرات فرکانس بدون ایجاد پرش، پالس ناقص یا اعوجاج انجام می‌شود. بنابراین هدف اصلی آزمایش، یعنی کنترل نرم و پیوسته فرکانس سیگنال PWM با حفظ Duty Cycle ثابت، با موفقیت محقق شد.

با فعال کردن ماژولهای تایمر، وقفه و ADC در میکروکنترلر، تمرین زیر را انجام دهید: یک سیگنال PWM ایجاد نمایید بطوریکه فرکانس آن در بازه 0Hz تا 1kHz و Duty Cycle آن در بازه 100٪ قابل تغییر باشد. ۲ عدد پتانسیومتر روی پایه‌ها ADC1_IN1 و ADC1_IN2 قرار دهید و با چرخاندن یکی از آنها، فرکانس را در بازه فوق تغییر دهید و با چرخاندن پتانسیومتر دیگر، Duty Cycle را در بازه 0 تا 100٪ تغییر دهید. تغییر Duty Cycle و فرکانس با تغییر موقعیت پتانسیومترها بایستی بصورت کاملاً یکنواخت صورت گیرد و نباید هیچ پرشی در مشخصات سیگنال تولیدی با اندکی دوران در موقعیت پتانسیومترها، رخ دهد. تغییر یک کمیت مانند Duty Cycle، نباید باعث تغییر پارامتر دیگر مانند فرکانس شود (وبالعکس).

اطلاعات کلی و محاسبات

هدف آزمایش

هدف این آزمایش، کنترل همزمان فرکانس و Duty Cycle یک سیگنال PWM با استفاده از دو پتانسیومتر مستقل است. در این پروژه، ولتاژ خروجی هر پتانسیومتر توسط یک کانال ADC اندازه‌گیری شده و هر یک وظیفه کنترل یکی از مشخصات سیگنال را بر عهده دارند.

- پتانسیومتر اول (PA1 / ADC1_IN1) فرکانس خروجی را در بازه 0 تا 1000 هرتز تنظیم می‌کند.
- پتانسیومتر دوم (PA2 / ADC1_IN2) نسبت سیکل کاری (Duty Cycle) را در بازه 0 تا 100 درصد تغییر می‌دهد.

یکی از مهم‌ترین اهداف این تمرین، مستقل بودن این دو پارامتر از یکدیگر است؛ به این معنا که تغییر فرکانس نباید باعث تغییر Duty Cycle شود و همچنین تغییر Duty Cycle نیز نباید بر فرکانس خروجی تأثیر بگذارد. علاوه بر این، تغییرات باید کاملاً نرم و بدون ایجاد پرش یا اعوجاج در سیگنال انجام شوند.

معرفی اجزای مورد استفاده

در این پروژه از واحدهای سخت‌افزاری زیر استفاده شده است:

- میکروکنترلر STM32F103C8
- کلاک داخلی HSI با فرکانس 8 MHz

- تایمر TIM1
 - مبدل آنالوگ به دیجیتال ADC1
 - کانال ADC1_IN1 پایه PA1
 - کانال ADC1_IN2 پایه PA2
 - دو عدد پتانسیومتر
 - پایه PA0 به عنوان خروجی PWM
- پایه PA1 تنها برای کنترل فرکانس و پایه PA2 تنها برای کنترل Duty Cycle استفاده می‌شود، بنابراین هر دو پارامتر کاملاً مستقل از یکدیگر خواهند بود.

نحوه عملکرد سیستم

- در هر تکرار از برنامه، ابتدا مقدار ولتاژ هر دو پتانسیومتر توسط ADC خوانده می‌شود.
- مقدار کانال اول (ADC1_IN1) برای تعیین فرکانس خروجی استفاده می‌شود.
 - مقدار کانال دوم (ADC1_IN2) برای تعیین Duty Cycle به کار می‌رود.
- پس از محاسبه فرکانس، مقدار رجیستر ARR به‌روزرسانی می‌شود. سپس مقدار CCR1 بر اساس ARR جدید و مقدار Duty Cycle محاسبه می‌شود. این روش باعث می‌شود که درصد Duty Cycle حتی با تغییر فرکانس نیز ثابت باقی بماند.
- همچنین با فعال بودن قابلیت **Auto-Reload Preload**، مقدار جدید ARR تنها پس از پایان سیکل جاری اعمال می‌شود و در نتیجه تغییر فرکانس بدون پرش انجام خواهد شد.

محاسبات فرکانس

برای ساده شدن محاسبات، مقدار Prescaler برابر 79 انتخاب شده است.
بنابراین فرکانس شمارش تایمر برابر خواهد بود:

$$f_{TIM} = \frac{8000000}{79 + 1} = 100000Hz$$

مقدار ADC فرکانس در بازه زیر قرار دارد:

$$0 \leq ADC_{Freq} \leq 4095$$

فرکانس خروجی از رابطه زیر به دست می‌آید:

$$f = \frac{ADC_{Freq} \times 1000}{4095}$$

از رابطه فرکانس تایمر خواهیم داشت:

$$ARR = \frac{100000}{f} - 1$$

که با حذف محاسبات اعشاری به صورت زیر نوشته می‌شود:

$$ARR = \frac{409500}{ADC_{Freq}} - 1$$

محاسبات Duty Cycle

مقدار پتانسیومتر دوم نیز در بازه 0 تا 4095 قرار دارد.

برای مستقل بودن Duty Cycle از فرکانس، مقدار CCR1 همواره بر اساس مقدار جدید ARR محاسبه می‌شود:

$$CCR1 = \frac{ADC_{Duty} \times (ARR + 1)}{4095}$$

به این ترتیب درصد Duty Cycle مستقل از مقدار فرکانس خواهد بود.

نمونه عملکرد سیستم

Duty Cycle فرکانس خروجی ADC Duty ADC فرکانس

1024	1024	حدود 250Hz	25%
2048	2048	حدود 500Hz	50%
3072	3072	حدود 750Hz	75%
4095	4095	1000Hz	100%

این جدول نشان می‌دهد که تغییر هر پتانسیومتر تنها پارامتر مربوط به خود را تغییر داده و پارامتر دیگر بدون تغییر باقی می‌ماند.

پیکربندی پروژه در STM32CubeMX

ابتدا پروژه جدیدی برای میکروکنترلر STM32F103C8Tx ایجاد شد.

تنظیم پایه‌های ADC

پایه PA1 به عنوان

ADC1_IN1

برای کنترل فرکانس انتخاب شد.

پایه PA2 نیز به صورت

ADC1_IN2

برای کنترل Duty Cycle تنظیم گردید.

تنظیم پایه خروجی

پایه PA0 به عنوان

GPIO Output

پیکربندی شد تا سیگنال PWM روی آن تولید شود.

فعال‌سازی ADC

در بخش ADC1، دو کانال زیر فعال شدند:

• ADC1_IN1

• ADC1_IN2

از آنجا که در این پروژه از DMA استفاده نشده است، هر دو کانال به صورت نرم‌افزاری و به نوبت نمونه‌برداری می‌شوند.

تنظیم تایمر TIM1

تنظیمات تایمر مشابه تمرین قبل انجام شد.

پارامترهای اصلی عبارت‌اند از:

• Clock Source = Internal Clock

• Counter Mode = Up

• Prescaler = 79

• Output Compare No Output

فعال کردن Auto-Reload Preload

گزینه

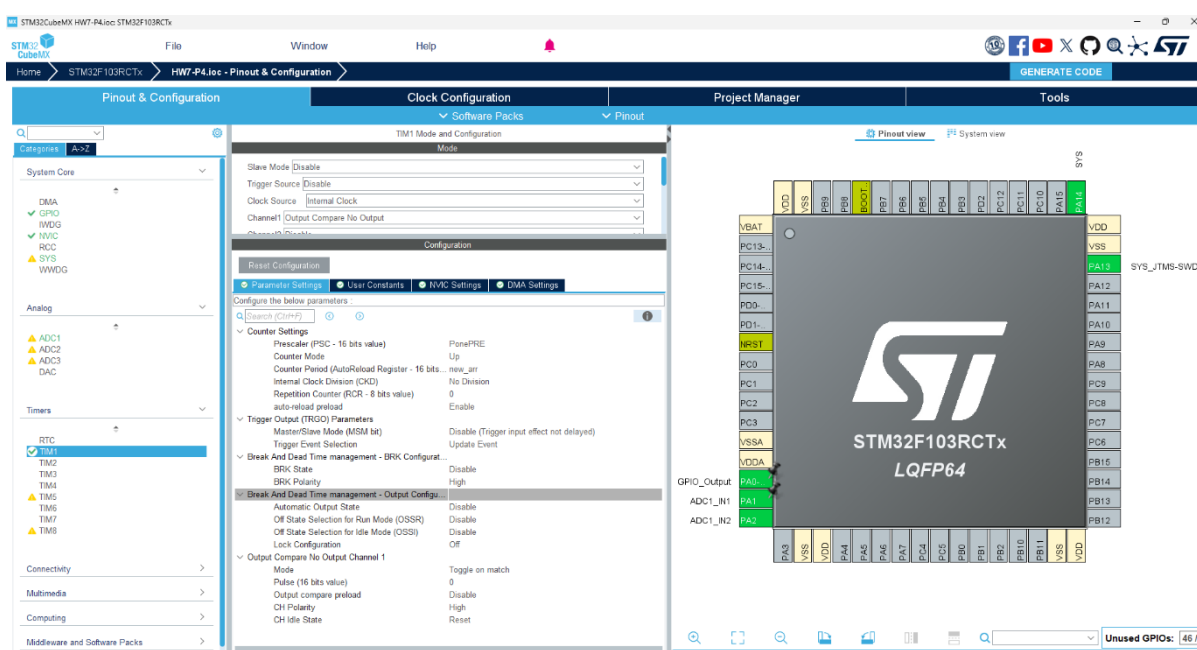
Auto Reload Preload

فعال شد.

این قابلیت باعث می‌شود مقدار جدید ARR پس از پایان سیکل جاری اعمال شده و تغییر فرکانس کاملاً نرم و بدون ایجاد پرش انجام شود.

فعال کردن وقفه‌ها

در بخش NVIC ، وقفه TIM1 فعال شد تا Callback های مربوط به Update و Output Compare اجرا شوند.



تولید پروژه

در پایان IDE روی Keil MDK-ARM تنظیم شد و پروژه توسط CubeMX تولید گردید.

پیاده‌سازی برنامه در Keil

پس از تولید پروژه، بخش‌های موردنیاز در فایل main.c تکمیل شدند.

تعریف متغیرها

```
uint16_t PonePRE = 79;  
uint32_t adc_freq = 0;
```

```
uint32_t adc_duty = 0;
uint32_t new_arr = 0;
uint32_t new_ccr = 0;
```

توضیح کد

در این بخش متغیرهای لازم برای ذخیره مقدار Prescaler ، مقادیر خوانده شده از دو کانال ADC ، مقدار جدید ARR و مقدار جدید CCR1 تعریف شده‌اند.

- **adc_freq** مقدار پتانسیومتر کنترل فرکانس را ذخیره می‌کند .
- **adc_duty** مقدار پتانسیومتر کنترل Duty Cycle را نگهداری می‌کند .
- **new_arr** برای محاسبه فرکانس جدید استفاده می‌شود .
- **new_ccr** مقدار جدید Duty Cycle را تعیین می‌کند .

تابع Read_ADC_Channel

```
uint32_t Read_ADC_Channel(uint32_t channel)
{
    ADC_ChannelConfTypeDef sConfig = {0};

    sConfig.Channel = channel;
    sConfig.Rank = ADC_REGULAR_RANK_1;
    sConfig.SamplingTime = ADC_SAMPLETIME_1CYCLE_5;

    HAL_ADC_ConfigChannel(&hadc1,&sConfig);

    HAL_ADC_Start(&hadc1);

    HAL_ADC_PollForConversion(&hadc1,10);

    return HAL_ADC_GetValue(&hadc1);
}
```

توضیح کد

از آنجا که تنها یک واحد ADC وجود دارد و از DMA استفاده نشده است، این تابع برای خواندن هر کانال به صورت جداگانه طراحی شده است.

ابتدا کانال موردنظر انتخاب می‌شود، سپس تبدیل ADC آغاز شده و پس از پایان تبدیل، مقدار دیجیتال آن کانال بازگردانده می‌شود.

به کمک این تابع می‌توان هر تعداد کانال ADC را تنها با ارسال شماره کانال خواند.

خواندن دو پتانسیومتر

```
adc_freq =  
Read_ADC_Channel(ADC_CHANNEL_1);  
  
adc_duty =  
Read_ADC_Channel(ADC_CHANNEL_2);
```

توضیح کد

ابتدا مقدار کانال اول (PA1) خوانده می‌شود که مسئول کنترل فرکانس است. سپس مقدار کانال دوم (PA2) دریافت می‌شود که برای تعیین Duty Cycle استفاده خواهد شد.

بررسی حالت صفر هرتز

```
if(adc_freq<5)  
{  
    HAL_TIM_Base_Stop_IT(&htim1);  
  
    HAL_TIM_OC_Stop_IT(&htim1,  
                        TIM_CHANNEL_1);  
  
    HAL_GPIO_WritePin(GPIOA,  
                       GPIO_PIN_0,  
                       GPIO_PIN_RESET);  
}
```

توضیح کد

اگر مقدار پتانسیومتر فرکانس تقریباً صفر باشد، تایمر متوقف شده و خروجی LOW می‌شود تا فرکانس صفر هرتز ایجاد گردد و از تقسیم بر صفر در محاسبات جلوگیری شود.

محاسبه فرکانس جدید

```
new_arr =  
(409500/adc_freq)-1;
```

توضیح کد

در این قسمت مقدار جدید ARR بر اساس مقدار ADC مربوط به فرکانس محاسبه می‌شود. هرچه مقدار پتانسیومتر بیشتر باشد، مقدار ARR کوچکتر شده و فرکانس افزایش می‌یابد.

محدود کردن مقدار ARR

```
if(new_arr>65535)
{
    new_arr=65535;
}
```

توضیح کد

از آنجا که رجیستر ARR یک رجیستر 16 بیتی است، مقدار آن نباید از 65535 بیشتر شود. این شرط از سرریز رجیستر جلوگیری می‌کند.

محاسبه Duty Cycle

```
new_ccr =
(adc_duty*
(new_arr+1))
/4095;
```

توضیح کد

این مهم‌ترین بخش برنامه است.

مقدار CCR1 بر اساس **ARR جدید** محاسبه می‌شود، نه یک مقدار ثابت. بنابراین اگر فرکانس تغییر کند، Duty Cycle همچنان درصد صحیح خود را حفظ خواهد کرد.

به عنوان مثال اگر Duty Cycle روی 30 درصد تنظیم شده باشد، با تغییر فرکانس همچنان 30 درصد باقی خواهد ماند.

اعمال تنظیمات تایمر

```
TIM1->ARR =
new_arr;

TIM1->CCR1 =
new_ccr;
```

توضیح کد

در پایان حلقه، مقدار جدید ARR و CCR1 داخل رجیسترهای تایمر قرار می‌گیرند.

به دلیل فعال بودن قابلیت Auto-Reload Preload ، مقدار ARR در انتهای سیکل جاری اعمال می‌شود و تغییر فرکانس بدون پرش انجام خواهد شد.

تأخیر برنامه

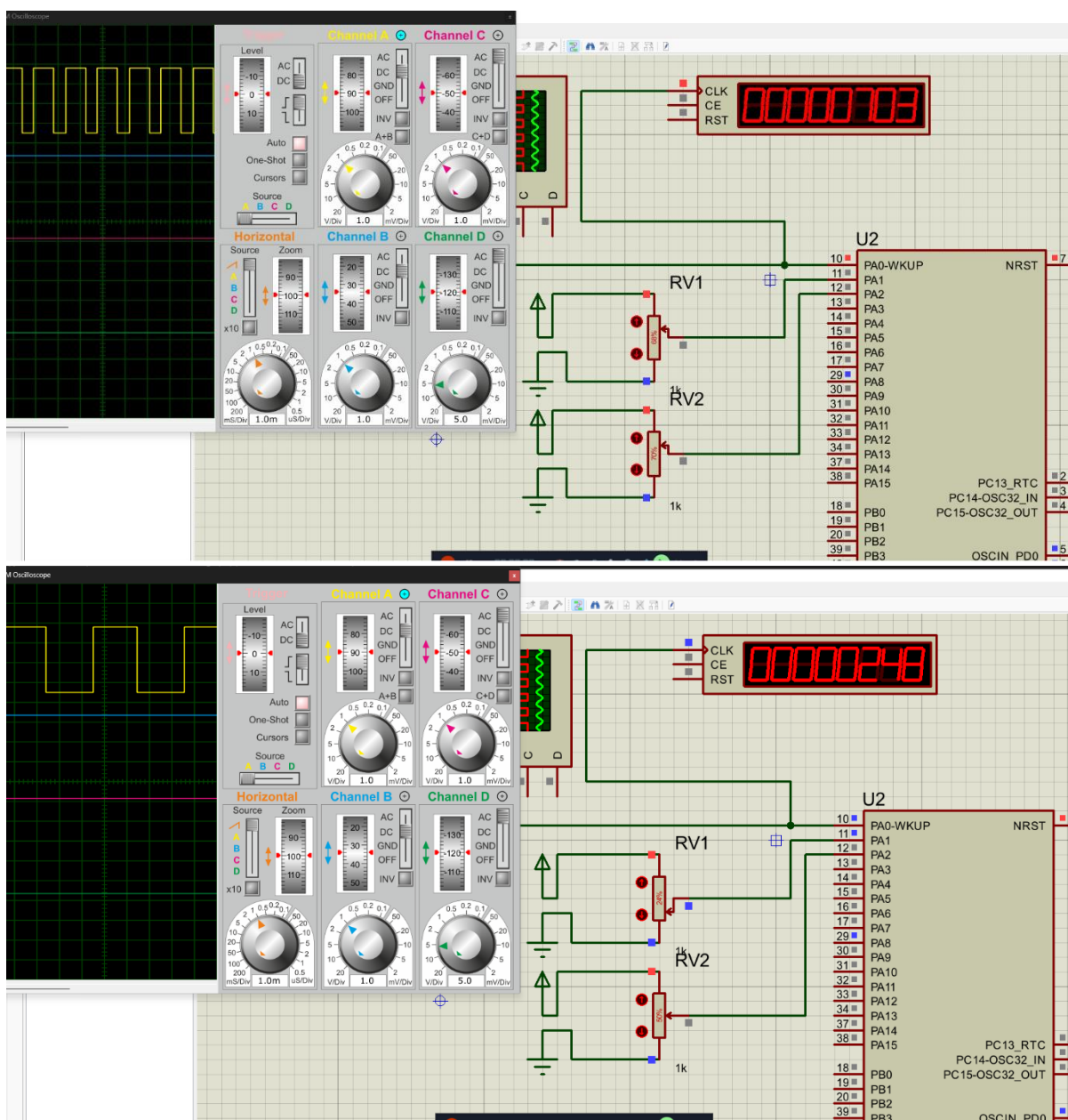
HAL_Delay(10);

توضیح کد

این تأخیر کوتاه موجب کاهش بار پردازنده، افزایش پایداری برنامه و عملکرد مناسب‌تر شبیه‌سازی در Proteus می‌شود.

شبیه‌سازی در Proteus

برای بررسی عملکرد مدار از نرم‌افزار Proteus استفاده شد.



قطعات مورد استفاده

در مدار از قطعات زیر استفاده شد:

- STM32F103C8
- دو عدد پتانسیومتر POT-HG
- Oscilloscope
- Frequency Counter
- منبع تغذیه 3.3 ولت
- Ground

نحوه اتصال مدار

دو سر هر پتانسیومتر به منبع تغذیه و زمین متصل شدند.

پایه میانی پتانسیومتر اول به **PA1 (ADC1_IN1)** و پایه میانی پتانسیومتر دوم به **PA2 (ADC1_IN2)** متصل شد. پایه **PA0** نیز به صورت همزمان به اسیلوسکوپ و Frequency Counter متصل گردید تا شکل موج، فرکانس و تغییرات آن قابل مشاهده باشد.

بارگذاری فایل برنامه

پس از کامپایل پروژه در Keil ، فایل HEX تولید شد و در قسمت **Program File** میکروکنترلر در Proteus بارگذاری گردید. فرکانس کاری میکروکنترلر نیز روی **8MHz** تنظیم شد.

تحلیل نتایج

با اجرای شبیه‌سازی مشاهده شد که چرخاندن پتانسیومتر متصل به **PA1** تنها باعث تغییر فرکانس سیگنال خروجی در بازه **0 تا 1000 هرتز** می‌شود، در حالی که Duty Cycle بدون تغییر باقی می‌ماند. همچنین تغییر پتانسیومتر متصل به **PA2** تنها عرض پالس خروجی را از **0 تا 100 درصد** تغییر داده و هیچ تأثیری بر فرکانس ندارد.

بررسی شکل موج در اسیلوسکوپ و مقدار نمایش داده‌شده توسط شمارنده فرکانس نشان داد که هر دو پارامتر به صورت **کاملاً مستقل** کنترل می‌شوند و به دلیل استفاده از قابلیت **Auto-Reload Preload**، تغییرات فرکانس بدون ایجاد پرش یا اعوجاج در سیگنال انجام می‌شود. بنابراین هدف اصلی آزمایش، یعنی کنترل همزمان و مستقل فرکانس و Duty Cycle سیگنال PWM با استفاده از دو کانال ADC ، با موفقیت محقق شد.

تمرین فوق را به جای فعال کردن مازول ADC، با فعال کردن مازول UART و ارسال داده از نرم افزار Matlab، انجام دهید. دقت داشته باشید که اعداد ارسالی از Matlab، برای تغییر Duty Cycle، بایستی بصورت ۱۲ بیتی (بازه 0 تا 4095) باشد همچنین اعداد ارسالی از Matlab، برای تغییر فرکانس بایستی در بازه 0 تا ۱۰۰۰ باشد (قابلیت تغییر فرکانس با دقت 1Hz). در این تمرین، بایستی میکروکنترلر متوجه باشد که کدام عدد ارسالی، برای تغییر فرکانس است و کدام عدد ارسالی برای تغییر Duty Cycle می باشد.

کلیات، محاسبات و منطق برنامه

هدف این پروژه، حذف پتانسیومترهای فیزیکی ADC و جایگزینی آنها با یک ارتباط سریال (UART) برای کنترل همزمان و مستقل «فرکانس» (در بازه ۰ تا ۱۰۰۰ هرتز) و «دیوتی سیکل» (در بازه ۰ تا ۴۰۹۵) سیگنال PWM است. سیگنال خروجی به صورت کاملاً نرم افزاری با استفاده از دو وقفه تایمر تولید می شود.

چالش ارسال ۱۲ بیت در گذرگاه ۸ بیتی

از آنجا که پورت UART به صورت استاندارد بسته های ۸ بیتی ارسال می کند و دیتای ما ۱۲ بیتی است (مقادیر تا ۴۰۹۵)، استفاده از ارسال رشته ای (String) روشی غیر استاندارد محسوب می شود. لذا یک پروتکل باینری خام ۲ بیتی (۱۶ بیت) طراحی شد.

- **۴ بیت پرارزش (Command):** مشخص می کند داده مربوط به کدام پارامتر است (عدد ۱ برای فرکانس، عدد ۲ برای دیوتی سیکل).
- **۱۲ بیت کم ارزش (Payload):** مقدار عددی پارامتر.

متلب این دو بایت (High Byte و Low Byte) را با استفاده از عملیات شیفت بیتی (bitshift) ساخته و ارسال می کند. میکروکنترلر نیز در وقفه UART، این دو بایت را دریافت کرده و با استفاده از عملگرهای & و | و << داده ۱۲ بیتی را بازسازی می کند.

محاسبات تایمر

کلاک میکرو ۸ مگاهرتز است. با قرار دادن:

$$Prescaler = 79$$

کلاک تایمر روی

$$100,000 \text{ Hz}$$

تنظیم می‌شود.

محاسبه فرکانس:

$$ARR = \frac{100000}{Target_Freq} - 1$$

محاسبه دیوتی سایکل (مستقل از فرکانس):

$$CCR1 = \frac{Target_Duty \times (ARR + 1)}{4095}$$

تنظیمات STM32CubeMX

پیکربندی سخت‌افزاری میکروکنترلر در محیط CubeMX به این شرح انجام گرفت.

بخش System Core

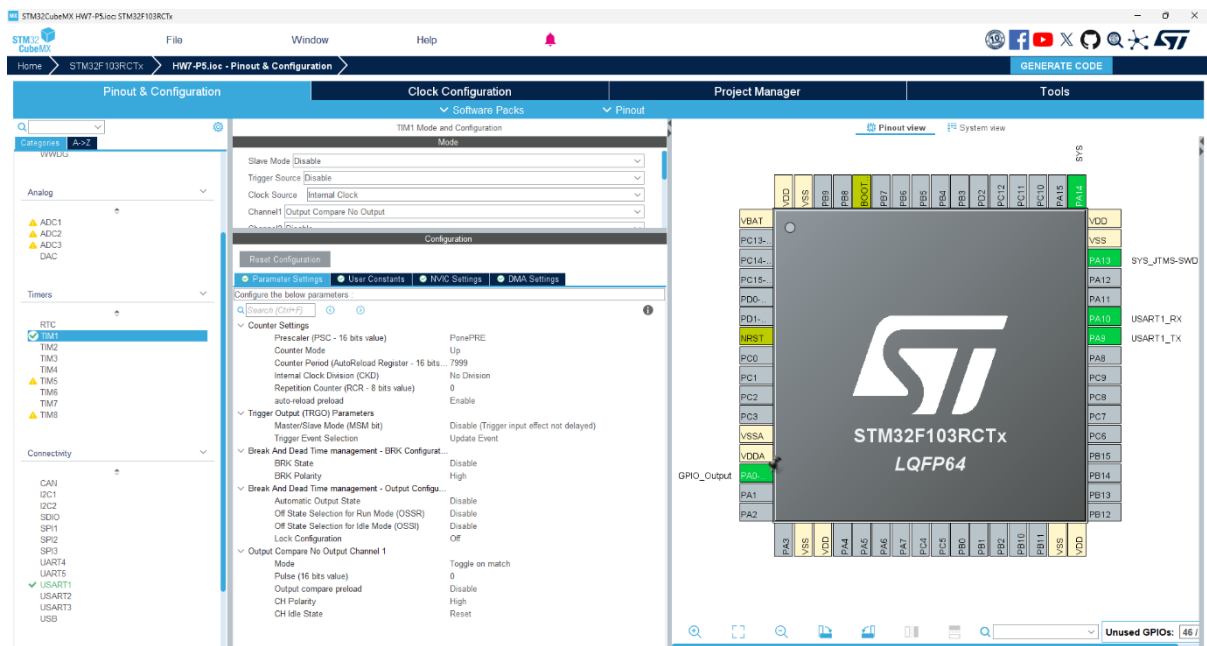
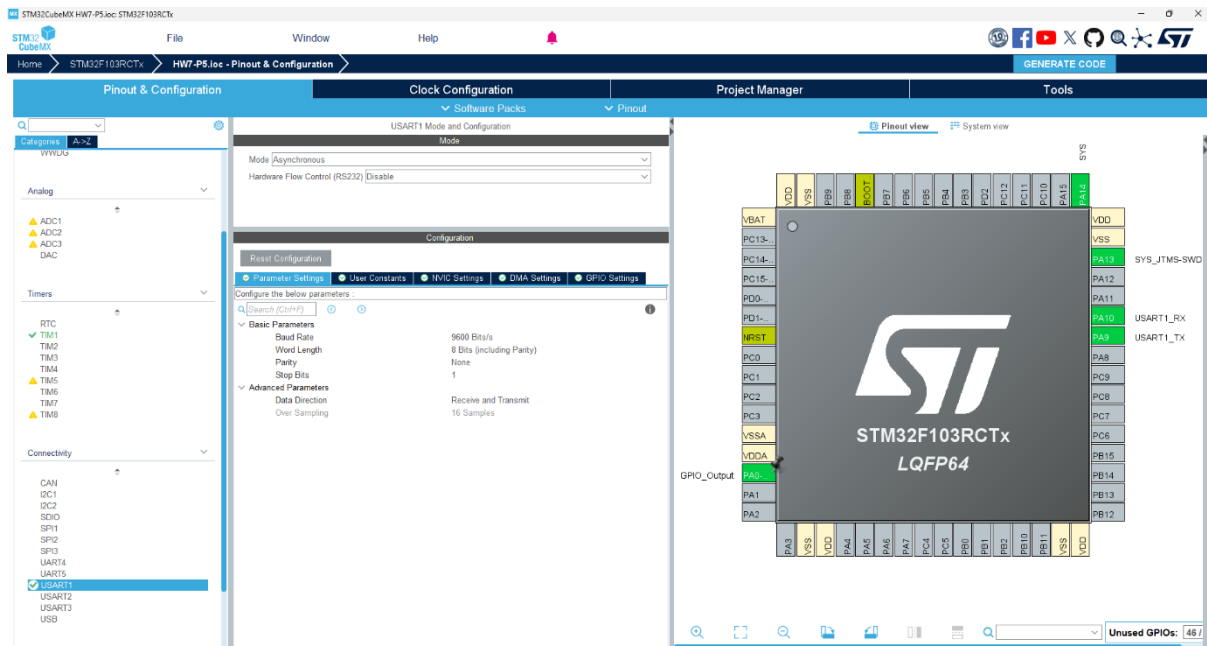
کلاک سیستم روی **Internal Clock (HSI)** با فرکانس ۸ مگاهرتز تنظیم شد. پین **PA0** به عنوان **GPIO_Output** (خروجی PWM) تعریف گردید.

بخش Timers (TIM1)

- منبع کلاک روی **Internal Clock** و **Channel 1** روی **Output Compare No Output** قرار گرفت.
- مقدار **Prescaler = 79** و مقدار **ARR** روی یک عدد پیش‌فرض مانند **7999** تنظیم شد (این عدد در کد اوررایت می‌شود).
- قابلیت **Auto-Reload Preload** برای جلوگیری از پرش فرکانس فعال (Enable) شد.
- در تب **NVIC**، تیک دو وقفه **TIM1 Update Interrupt** و **TIM1 Capture Compare Interrupt** زده شد.

بخش Connectivity (USART1)

حالت روی **Asynchronous** با **Baud Rate = 9600** (۸ بیت داده، بدون Parity) تنظیم شد.
در تب **NVIC** این بخش، وقفه **USART1 Global Interrupt** فعال گردید تا میکرو بتواند دیتای متلب را در پس‌زمینه دریافت کند.



۳. کدنویسی در Keil و MATLAB

کد Keil

در این کد، وقفه سریال دقیقاً منتظر ۲ بایت می‌ماند، دیتا را استخراج کرده و سپس در حلقه while مقادیر جدید به رجیسترهای تایمر اعمال می‌شوند.

```
/* USER CODE BEGIN PV */
uint16_t PonePRE = 79;
uint32_t target_freq = 50;
uint32_t target_duty = 2048;
```

```

uint8_t rx_buffer[2];
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        uint8_t command = (rx_buffer[0] >> 4) & 0x0F;

        uint16_t value = ((rx_buffer[0] & 0x0F) << 8) | rx_buffer[1];

        if (command == 1)
            target_freq = value;

        else if (command == 2)
            target_duty = value;
        HAL_UART_Receive_IT(&huart1, rx_buffer, 2);
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (TIM1->CCR1 > 0)
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_SET);
    }
}

void HAL_TIM_OC_DelayElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_RESET);
    }
}
/* USER CODE END 0 */

int main(void)
{
    HAL_Init();

```

```

SystemClock_Config();
MX_GPIO_Init();
MX_TIM1_Init();
MX_USART1_UART_Init();

/* USER CODE BEGIN 2 */
TIM1->PSC = PonePRE;

HAL_UART_Receive_IT(&huart1, rx_buffer, 2);
/* USER CODE END 2 */

while (1)
{
    if (target_freq == 0)
    {
        HAL_TIM_Base_Stop_IT(&htim1);
        HAL_TIM_OC_Stop_IT(&htim1, TIM_CHANNEL_1);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_RESET);
    }
    else
    {
        HAL_TIM_Base_Start_IT(&htim1);
        HAL_TIM_OC_Start_IT(&htim1, TIM_CHANNEL_1);

        uint32_t new_arr = (100000 / target_freq) - 1;
        if (new_arr > 65535)
            new_arr = 65535;

        uint32_t new_ccr = (target_duty * (new_arr + 1)) / 4095;

        TIM1->ARR = new_arr;
        TIM1->CCR1 = new_ccr;
    }

    HAL_Delay(10);
}
}

```

کد MATLAB

این اسکریپت دیتای ۱۲ بیتی را شکسته، پروتکل را ایجاد کرده و برای مشاهده تغییرات پیوسته، مقادیر را به صورت تدریجی (Sweep) ارسال می‌کند.

```
try
    clear s;
catch
end

s = serialport("COM3",9600);

disp('--- Starting Raw Binary Hardware Test Sequence ---');

write(s,[bitor(bitshift(1,4),bitshift(50,-8)),bitand(50,255)],"uint8");
write(s,[bitor(bitshift(2,4),bitshift(2048,-8)),bitand(2048,255)],"uint8");

pause(3);

disp('Sweeping Frequency Up...');
for freq=50:25:1000
    write(s,[bitor(bitshift(1,4),bitshift(freq,-8)),bitand(freq,255)],"uint8");
    pause(0.4);
end

pause(3);

disp('Sweeping Duty Cycle...');

for duty=2048:-80:200
    write(s,[bitor(bitshift(2,4),bitshift(duty,-8)),bitand(duty,255)],"uint8");
    pause(0.3);
end

pause(2);

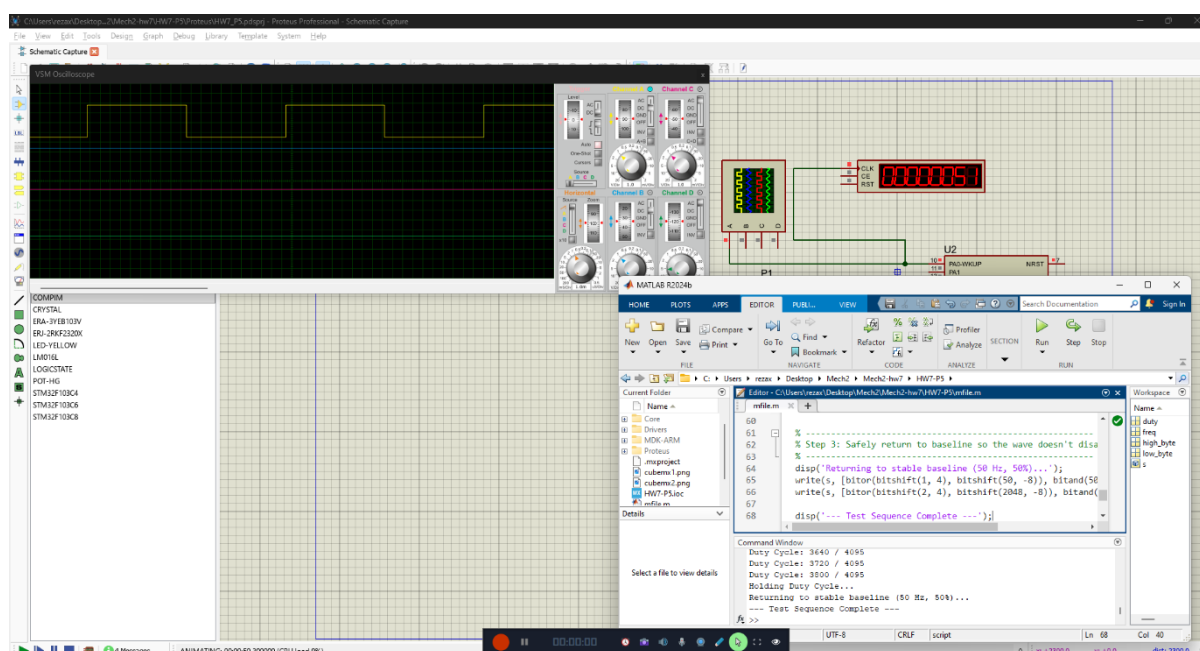
for duty=200:80:3800
    write(s,[bitor(bitshift(2,4),bitshift(duty,-8)),bitand(duty,255)],"uint8");
    pause(0.3);
end

pause(3);
```

```
write(s,[bitor(bitshift(1,4),bitshift(50,-8)),bitand(50,255)],"uint8");
write(s,[bitor(bitshift(2,4),bitshift(2048,-8)),bitand(2048,255)],"uint8");

disp('--- Sequence Complete ---');
```

شبیه‌سازی در Proteus



برای اتصال کد متلب به میکروکنترلر در محیط شبیه‌ساز پروتئوس مراحل زیر انجام شد.

تنظیمات میکروکنترلر

فایل **hex** تولید شده روی قطعه **STM32F103C8** لود شد و فرکانس کاری (**OSC Frequency**) صریحاً روی **8 MHz** تنظیم گردید.

تنظیمات پورت سریال (COMPIM)

ماژول **COMPIM** برای اتصال پورت مجازی **COM2**

Baud Rate فیزیکی و مجازی هر دو روی **9600** تنظیم شدند.

نکته کلیدی برای جلوگیری از فریز شدن متلب، قرار دادن گزینه **Hardware Flow Control** روی **None** بود.

پایه **TXD** از ماژول به پایه **PA10 (RX)** میکرو و پایه **RXD** از ماژول به پایه **PA9 (TX)** متصل شد.

تجهیزات اندازه‌گیری

یک اسیلوسکوپ و یک فرکانس‌متر دیجیتال (در حالت **Operating Mode: Frequency**) به پایه **PA0** متصل گردید.

نتیجه نهایی

پس از اجرای شبیه‌سازی در Proteus و Run کردن اسکریپت متلب، اسیلوسکوپ بدون هیچ‌گونه پرش یا توقف، تغییرات پیوسته فرکانس و عرض پالس (همگام با حلقه‌های for متلب) را به صورت زنده نمایش داد. تغییر فرکانس هیچ تأثیری بر روی نسبت دیوتی ساینکل نگذاشت که این امر صحت عملکرد منطق ریاضی سیستم را تأیید می‌کند.